Wayne C.
Published Nov 2, 2025

+ Follow

Introduction

In mission-critical environments, database availability is non-negotiable. PostgreSQL has evolved from being a rock-solid single-node database into a powerful, resilient system capable of handling tools available (HA) needs through automation. failover, switchover and fault-tolerance and recovery workflows to support always-on architectures.

1. Understanding High Availability Concepts

- **Failover**: Automatic or manual switch from a failed primary to a standby server.
- **Switchover**: Planned switch between primary and standby, often for maintenance.
- **Node Rejoining**: Reintegrating an old or failed node back into the cluster after failure or degrade.

These operations are foundational in ensuring business continuity and minimizing downtime.

2. PostgreSQL Replication Primer

Before understanding failover/switchover, you need replication:

- **Streaming Replication**: PostgreSQL native synchronous or asynchronous data streaming.
- **Logical Replication**: For more flexible, table-level replication.
- **Tools**: repmgr, pg_auto_failover, Patroni, pgBackRest, barman, Stolon.

3. Failover Explained

Failover occurs when the primary server becomes unavailable.

Automatic Failover:

Tools like:

- **pg_auto_failover**
- **Patroni (with Etcd or Consul)**
- **Stolon**

Trigger promotions of standby nodes with heartbeat monitoring.

Manual Failover:

1. Detect failure
2. Promote standby: pg_ctl promote
3. Redirect applications to new primary
4. Reconfigure old primary as standby

4. Switchover: Controlled Role Swap

Use case: Patching the primary server.

Steps:

1. Ensure all WALs are synced to standby
2. Promote standby using pg_ctl promote or tool-specific commands.
3. Demote primary cleanly.
4. Update application connection strings or proxy (e.g. PgBouncer).

Best Tools:

- repmgr standby switchover
- Patroni's switchover command

5. Node Rejoining: Reintegration After Failure

Scenario:

After failover, the old primary is outdated or inconsistent.

Approaches:

- **Rebuild node:**
- **Rejoin using repmgr:**
- **Auto-Rejoin with pg_auto_failover**: It auto handles this with minimal input.

Cautions:

- Rejoining without resync can cause split-brain.
- Always ensure full data sync.

6. Configuration Patterns

Primary/Standby with WAL Archiving:

- Enables Point-in-Time Recovery (PITR)
- Ensures old nodes can recover if WALs retained

Synchronous Standby Setup:

In postgresql.conf:

```
synchronous_standby_names = 'FIRST 1 (standby1, standby2)'
```

Enables failover without data loss.

7. Proxy & Connection Management

Use tools like:

- **PgBouncer** or **HAProxy**
- **pgbouncer-rr** (read/write routing)

They enable smooth failover/switchover by abstracting the database hostname/IP.

8. Monitoring & Alerts

Monitor:

- Replication lag
- WAL activity
- Node health

Tools:

- **Prometheus + Grafana**
- **pg_stat_replication**
- **pg_autoctl show state**

9. Troubleshooting Failover Events

10. Best Practices for HA with PostgreSQL

- Always use WAL archiving and retention.
- Monitor disk usage — WAL growth can fill disks fast.
- Automate failover but notify operators.
- Always test failover/switchover in staging.

11. Real-World Use Cases

- **FinTech**: Use Patroni with Consul for ultra-fast failover (<10s)
- **E-commerce**: Use pg_auto_failover to support microservice-ter auto promotion
- **SaaS**: Repmgr for manual switchover during version upgrades

Patroni is a HA framework that automates PostgreSQL failover using a distributed configuration store (DCS) like Consul or Etcd. When paired with **Consul**, it enables leader election, cluster state management, and **ultra-fast failover (typically under 10 seconds)**.

🧭 **Architecture**

⚙️ **Components**

- **PostgreSQL 15+**
- **Patroni** (Python-based)
- **Consul** (for DCS/leader election)
- **HAProxy / PgBouncer** (for routing)
- **Optional**: vip-manager, DNS, or Keepalived for client redirection

🔧 **Installation & Configuration Steps**

1. Install Consul on all nodes

```
sudo apt install consul
```

Basic Consul config (/etc/consul.d/consul.json):

```
{
  "datacenter": "dc1",
  "data_dir": "/opt/consul",
  "server": false,
  "retry_join": ["provider=aws,tag_key=Consul,tag_value=server"]
}
```

Start Consul:

```
consul agent -config-dir=/etc/consul.d
```

2. Install Patroni on each DB node

```
sudo apt install python3-pip
pip3 install patroni[consul]
```

3. Configure Patroni (example: /etc/patroni.yml)

```
scope: postgres-cluster
name: node1

restapi:
  listen: 0.0.0.0:8008
  connect_address: 10.0.0.1:8008

etcd:
  host: 127.0.0.1:2379 # For Consul use:
  protocol: http
  host: 127.0.0.1:8500
  register_service: true

postgresql:
  listen: 0.0.0.0:5432
  connect_address: 10.0.0.1:5432
  data_dir: /var/lib/postgresql/data
  authentication:
    replication:
      username: replicator
      password: replicpass
    superuser:
      username: postgres
      password: postgres
  parameters:
    wal_level: replica
    hot_standby: "on"

tags:
  nofailover: false
  noloadbalance: false
```

4. Start Patroni on each node

```
patroni /etc/patroni.yml
```

On first boot:

- Initializes cluster
- Consul holds the lock for leader
- Remaining nodes stay in standby mode

5. Optional: HAProxy for Client Connections

haproxy.cfg example:

```
frontend pgsql_front
  bind *:5000
  default_backend pgsql_back

backend pgsql_back
  option tcp-check
  server node1 10.0.0.1:5432 check port 8008
  server node2 10.0.0.2:5432 check port 8008
```

🔄 **How Failover Works**

1. Leader node crashes (network, kernel panic, etc.)
2. Consul detects it via TTL health check
3. Patroni on standby nodes takes the Consul leader lock
4. Standby is promoted to primary
5. HAProxy re-routes traffic to the new primary
6. Downtime: <10s

🧪 **Monitoring Commands**

- Cluster state:
- From Consul:

🧠 **Best Practices**

- Always use TLS for Consul and PostgreSQL
- Monitor pg_stat_replication, Patroni REST API, and Consul health checks
- Enable WAL archiving + base backups (for PITR)
- Use pgbackrest or barman for disaster recovery

🚀 **Real-World Impact**

A fintech platform reduced RTO from **5 minutes to 7 seconds** *using this setup with zero manual intervention. Combined with GitOps-managed configs and PgBouncer pools, they achieved 99.99% DB uptime.*

Conclusion

PostgreSQL provides a rich and flexible toolkit for building robust, highly available database systems. Whether you're using pg_auto_failover, repmgr, or Patroni, mastering failover, switchover, and node rejoining ensures your PostgreSQL infrastructure can survive disasters and thrive under continuous operations.

To view or add a comment, sign in

More articles by Wayne C.

Nov 2, 2025
PostgreSQL - Automatic Failover with Auto Failover Extensio...
1. Introduction PostgreSQL has become a cornerstone for many mission-critical applications, but high availability (HA)...
1

Nov 3, 2025
PostgreSQL – Warm Standby & Streaming Replication
PostgreSQL provides robust warm standby and streaming replication capabilities, allowing you to build high-availability...
1

Nov 3, 2025
PostgreSQL Replication & High Availability Checklist
🔧 Core Configuration Enable wal_level = replica (or logical for logical replication) Set max_wal_senders to a...

Nov 3, 2025
PostgreSQL - Replication & High Availability
1. Introduction High availability (HA) and replication are the backbone of mission-critical systems.

Show more

Explore content categories

Career Productivity Finance Soft Skills & Emotional Intelligence
Project Management Education
Show more

© 2025
About User Agreement
Accessibility Cookie Policy
Privacy Policy Brand Policy
Guest Controls Community Guidelines
Language